

Architecture et Programmation Web

Séance 8 : Authentification et Sessions

Nicolas Burny

9 avril 2026

ECAM

Introduction

Le problème : HTTP est sans état (*Stateless*)

- **Indépendance** : Chaque requête HTTP est traitée de manière isolée par le serveur.
- **Absence de mémoire** : Le protocole ne conserve aucune information d'une requête à l'autre.
- **Analogie** : C'est comme discuter avec quelqu'un qui oublierait qui vous êtes entre chaque phrase.

La problématique

Comment identifier qu'une requête sur `/profile` provient du même utilisateur qui s'est authentifié sur `/login` l'instant d'avant ?

- **Cookies** : Petits fichiers stockés chez le client (navigateur).
- **Sessions** : Mécanisme pour stocker des données côté serveur liées à un utilisateur spécifique.
- En Flask, la session est stockée dans un cookie **signé** côté client.

Gérer les Sessions avec Flask

Utiliser l'objet session

```
from flask import Flask, session, redirect, url_for

app = Flask(__name__)
app.secret_key = 'une_cle_tres_secrete' # Crucial pour la signature

@app.route('/login')
def login():
    session['user_id'] = 123
    return "Connecté !"

@app.route('/logout')
def logout():
    session.pop('user_id', None)
    return "Déconnecté !"
```

- **Signature cryptographique** : Flask signe le cookie avec `SECRET_KEY`.
- L'utilisateur peut voir la donnée, mais s'il la modifie, la signature devient invalide.
- **Attention** : Ne jamais stocker de données sensibles (mots de passe) directement en session. Tout est lisible par l'utilisateur.

Gestion des Mots de Passe

La règle d'or : Ne jamais stocker en clair

- Si votre base de données est compromise (fuite), tous les mots de passe sont révélés.
- Beaucoup d'utilisateurs réutilisent le même mot de passe partout.
- **Solution** : Le Hachage (Hashing).

Propriétés Fondamentales

- **Sens unique** : Impossible de retrouver l'entrée à partir du hash.
- **Déterministe** : Le même texte donne toujours le même hash.
- **Taille fixe** : La sortie a une longueur constante, peu importe l'entrée.

Le "Grain de Sel" (Salt)

- Ajout d'une chaîne aléatoire au mot de passe avant hachage.
- **But** : Rendre le hash unique pour chaque utilisateur (même si MDP identique).
- **Protection** : Invalide les *Rainbow Tables* (attaques pré-calculées).

Pourquoi utiliser un "Sel" (Salt) ?

Le sel est une défense critique pour transformer une attaque de masse en attaque individuelle :

- **Contre les Rainbow Tables** : Les pirates utilisent des fichiers géants de hashes pré-calculés. Le sel rend ces fichiers inutilisables car le hash dépend maintenant d'une valeur aléatoire.
- **Même mot de passe $\neq$ Même hash** : Si deux utilisateurs utilisent "123456", leurs hashes seront totalement différents. Personne ne peut deviner qu'ils partagent le même secret.
- **Coût de l'attaque** : Sans sel, craquer 10 000 comptes ne prend pas plus de temps qu'en craquer un seul. Avec le sel, l'effort du pirate est multiplié par le nombre d'utilisateurs.

Où est stocké le Sel ?

Le sel n'est pas un secret, il est stocké en clair avec le hash.

- **Format composite** : Les bibliothèques (comme Werkzeug) concatènent tout dans une seule chaîne de caractères.
- **Structure type** : `méthode:itérations$sel$hash`

Exemple concret de hash stocké en DB

`pbkdf2:sha256:260000$uV7b7XqI8jK$d9f2e...`

- **Vérification** : Lors du login, le programme extrait le sel (`uV7b7XqI8jK`), hache le mot de passe fourni avec lui, et compare le résultat !

Hachage avec Werkzeug

```
from werkzeug.security import generate_password_hash, check_password_hash

# 1. Lors de l'inscription : on transforme le mot de passe en hash
password = "mon_mot_de_passe_secret"
hash_a_stocker = generate_password_hash(password)
# On enregistre 'hash_a_stocker' dans la base de données

# 2. Lors de la connexion : on compare le hash stocké avec la saisie
saisie = "mon_mot_de_passe_secret"
est_valide = check_password_hash(hash_a_stocker, saisie) # Renvoie True

if est_valide:
    print("Connexion réussie !")
```

L'extension Flask-Login

Pourquoi Flask-Login ?

- Gère automatiquement la connexion/déconnexion.
- Fournit l'objet `current_user` accessible partout (même dans les templates).
- Gère la persistance ("Se souvenir de moi").
- Offre des outils pour protéger les routes.

Pour fonctionner avec Flask-Login, la classe `User` doit implémenter :

- `is_authenticated` : True si l'utilisateur est connecté.
- `is_active` : True si le compte est actif.
- `is_anonymous` : False pour les utilisateurs réels.
- `get_id()` : Retourne l'identifiant unique de l'utilisateur (en `str`).

Exemple de Modèle User

```
from flask_login import UserMixin

# UserMixin fournit les implémentations par défaut (is_authenticated, etc.)

class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True)
    password_hash = db.Column(db.String(128))
```

```
from flask_login import login_required, current_user

@app.route('/dashboard')
@login_required
def dashboard():
    return f"Bonjour {current_user.username} !"
```

Si l'utilisateur n'est pas connecté, il est renvoyé vers la page de login définie par :

```
login_manager.login_view = 'auth.login'
```

Conclusion

1. Utilisateur remplit le formulaire.
2. Serveur vérifie l'existence de l'utilisateur.
3. Serveur compare le hash du mot de passe fourni avec celui en DB.
4. Si OK : `login_user(user)`.
5. Flask-Login crée la session.

Authentification vs Autorisation

- **Authentification** : Identifier l'utilisateur (Qui est-ce ?) via Flask-Login.
- **Autorisation** : Restreindre l'accès aux ressources via `@login_required`.

Sécurité & Bonnes Pratiques

- **Hachage** : Ne jamais stocker de mots de passe en clair (`werkzeug.security`).
- **Session** : Utiliser `current_user` pour accéder aux données de l'utilisateur connecté.

Questions ?